

Optimization-Driven Quantum Circuit Reduction

Arnav Kapoor

August 15, 2026

Abstract

This report reproduces and extends *Optimization driven quantum circuit reduction* by Rosenhahn, Osborne and Hirche (New J. Phys. 27, 104509, 2025) [1]. We reimplement the paper’s local term-replacement scheme (variants V1–V3) in Python and add an *exact* engine built on a bit-exact symplectic (signed-tableau) lookup, which replaces the paper’s $1e-5$ numeric tolerance with equality up to global phase by construction. Running the paper’s own benchmark protocol (Tables 6/7, 100 random 4-qubit circuits of length 300, identical inputs for every method) we find the following. On the ion-trap gate set we *beat* the paper’s reported “Ours”: a mean of 71.6 ± 11.8 gates (cost-aware) or 73.5 ± 14.4 (length-minimising) versus the paper’s 111, and 27.2 RXX gates versus the paper’s 43; both differences are statistically significant ($p < 10^{-45}$, one-sample t -test, $n = 100$; Section 1). On the NISQ gate set a gap remains (160.5 vs. the paper’s 107). The inputs are composition-matched to the paper, yet we are able to *rule out* input composition, database depth, time budget and the paper’s own random-sampling loop as explanations. We further introduce a dependency-graph based sweep extension (*DAG block-compaction*) not present in the paper, and cross-check the NISQ diagnosis directly against the published method, ruling out two further candidate explanations; the residual gap is isolated to database factorization coverage on 3-wire blocks. All results reproduce on committed scripts and data.

1 Summary of verdicts

Table 1 summarises the head-to-head results against the paper’s reported “Ours”. Every difference is statistically significant: a one-sample t -test of the 100 per-circuit results against the paper’s reported mean gives $t = -26.0$ ($p = 4.8 \times 10^{-46}$, Cohen’s $d = -2.6$, 95% CI [70.6, 76.3]) for `exact_len` on ion trap and $t = -33.2$ ($p = 1.6 \times 10^{-55}$, $d = -3.3$, CI [69.2, 74.0]) for `exact_cost`; on NISQ, $t = +41.8$ ($p = 1.2 \times 10^{-64}$, $d = 4.2$, CI [158.3, 163.4]) for `numeric_len` and $t = +39.7$ ($p = 1.3 \times 10^{-62}$, $d = 4.0$, CI [157.8, 163.2]) for `numeric_cost`.

Table 1: Summary of verdicts against the paper’s reported “Ours”.

Benchmark	Paper “Ours”	Our best	Δ	Verdict
Ion trap, total gates	111	71.6	−39.4	WIN
Ion trap, two-qubit (RXX)	43	27.2	−15.8	WIN
NISQ, total gates	107	160.5	+53.5	gap
NISQ, two-qubit (CZ)	43	49.6	+6.6	gap
NISQ, deep, total	107	157.2	+50.2	narrower gap

2 Introduction

The paper’s objective is to shorten transpiled quantum circuits by a *local term replacement* scheme: a contiguous block of gates is sampled, its unitary is matched against a precomputed

compute graph of equivalent (shorter) circuits, and the block is substituted whenever a shorter equivalent exists up to global phase. Three variants are proposed: V1 (local replacement on the full database), V2 (random sub-block sampling against a wire-count indexed database) and V3 (V2 with a machine-learned gate that skips unpromising lookups).

This project has four goals:

1. reproduce the paper’s algorithm faithfully and validate the numeric protocol (strict 10^{-5} vs. loose 10^{-3} tolerance);
2. replace the tolerance-based engine with a *bit-exact* symplectic (Clifford) engine, removing tolerance fragility and making equivalence exact;
3. add a *cost-aware* objective (minimise two-qubit count, then length) — the decoherence-relevant direction the paper lists as future work;
4. run the paper’s own Tables 6/7 benchmark head-to-head against its reported numbers on identical circuits, with qiskit baselines and a baseline-fidelity check.

3 The paper’s method

A circuit is parsed into *gate tokens* on a fixed qubit register. The optimiser samples local blocks, remaps the active wires to local coordinates, and queries a precomputed compute graph: a BFS over the token pool in which nodes are phase-normalised unitaries, edges are pool gates, and each node stores the shortest token chain that realises it. A replacement is accepted only when it is strictly shorter and the unitary is preserved. The search is driven by an iteration budget in V1/V2 and by a time budget in our benchmarks. The two gate sets of interest are the ion-trap pool (RX, RY, RZ at $\pm\pi/2$; RXX at $\pi/2$; all Clifford) and the NISQ pool (RX, RZ at $\pm\pi/2, \pm\pi/4$; CZ; partially non-Clifford).

Figure 2 (Section 6) reproduces the paper’s Table 1 database-growth curve. The node count grows steeply with depth, which caps how deep a laptop-buildable lookup database can go.

4 Implementation

The repository is organised as a standard Python package (`qcr_repro` in `src/`) with benchmark and figure scripts under `scripts/`; the paper’s MATLAB demo is archived under `matlab_demo/`. Table 2 summarises the components.

Table 2: Repository components.

Module	Role
<code>src/symplectic.py</code>	bit-exact signed-tableau engine for Clifford pools (phase-preserving conjugation on Paulis)
<code>src/clifford.py</code>	binary-symplectic tableau engine (fast Clifford equivalence, $O(n^2)$ integer simulation)
<code>src/database.py</code>	compute-graph database (RAM backend; SQLite/disk backend for deep graphs)
<code>src/exact_database.py</code>	exact compute graphs for Clifford pools, Pareto-indexed by (two-qubit count, length)
<code>src/reducer.py</code>	numeric reducer: exhaustive sweeps, cost-aware objective, RZ-across-CZ pass, escape moves
<code>src/exact_reducer.py</code>	exact reducer over the symplectic database
<code>src/hybrid.py</code>	exact/numeric hybrid: Clifford windows solved exactly, others numerically
<code>src/rf_gate.py</code>	V3-style RF-gated lookup (online RandomForest, optional)
<code>src/prepass.py</code>	algebraic/ZX pre-passes (rotation fusion, CZ-CZ cancellation, RZ gathering)
<code>src/batched.py</code>	vectorised (batched) window-unitary sweep, bit-identical to the scalar sweep
<code>scripts/benchmark_comparison.py</code>	Tables 6/7 head-to-head benchmark
<code>scripts/generate_figures.py</code>	regenerates all figures

The pipeline follows the paper’s local-replacement concept:

1. parse the QASM circuit into gate tokens (angles snapped to the discrete pool so lookups are exact);
2. optional pre-passes (`src/prepass.py`) — adjacent same-axis rotation fusion, CZ·CZ cancellation, and RZ-gathering across CZ (NISQ) — all exact angle arithmetic, shrinking the input before search;
3. cluster single-qubit gates between two-qubit barriers and collapse 1-wire runs using the complete 1-wire graph;
4. exhaustive left-to-right window sweeps over blocks of length ≤ 8 against the compute-graph database;
5. transport shuffling (moving gates across gates they commute with) and equivalence-class *escape* moves that resample irreducible windows with structurally different equivalent words;
6. for NISQ, an RZ-across-CZ transport pass iterated to a fixpoint;
7. verify the output is unitary-equivalent to the input (exactly for Clifford pools, at 10^{-5} otherwise).

The exact engine keys every Clifford-pool lookup by a bit-exact signed tableau: per-qubit Pauli images are canonicalised with integer coefficients, so two circuits compare equal up to global phase with no floating point in the hot loop. The cost-aware variant minimises (two-qubit count, length) per replacement — including equal-length rewrites that remove RXX/CZ gates — which is the decoherence objective the paper defers to future work.

5 Verification infrastructure

Every reduction in this report is verified. Table 3 lists the automated checks shipped in the repository.

Table 3: Automated verification.

Check	What it verifies
<code>scripts/check_batched_vs_scalar.py</code>	the batched sweep is bit-identical to the scalar sweep; the pre-passes preserve the input unitary with only pool-representable gates
<code>scripts/check_levers.py</code>	the RF-gated lookup, exact/numeric hybrid and SQLite backend each preserve the input unitary (10^{-5}); the SQLite store is bit-identical to RAM
<code>benchmark_comparison.py</code> (built-in)	every reduced circuit is unitary-equivalent to its input

In all head-to-head runs below the equivalence pass rate is 1.000 (100/100 circuits), for both the exact and the numeric reducers.

6 Results

6.1 Demo-circuit protocol

The numeric port was first benchmarked with the paper’s protocol on its 5-qubit demo circuit (300 initial gates): depths $\{3, 4\}$, iteration budgets $\{500, 1000, 1500\}$, seeds $\{1, 5, 10\}$, under both strict (10^{-5}) and loose (10^{-3}) equivalence. Table 4 reports best and mean end lengths over the 3 seeds per configuration.

Table 4: Demo-circuit protocol: strict vs. loose tolerance (3 seeds per configuration).

depth	iters	runs	best _{strict}	mean _{strict}	eq _{strict}	best _{loose}	mean _{loose}	eq _{loose}
3	500	3	266	274.00	1.000	266	274.00	1.000
3	1000	3	246	250.00	0.667	246	250.00	1.000
3	1500	3	240	242.67	0.667	240	242.67	1.000
4	500	3	262	265.33	1.000	262	265.33	1.000
4	1000	3	238	243.33	1.000	238	243.33	1.000
4	1500	3	226	231.33	0.667	226	231.33	1.000

Best run: strict-valid 228 gates in 29.8 s (depth 4, 1500 iterations, seed 10); loose-valid 226 gates in 2.4 s. The loose metric accepts more replacements (eq. rate 1.000 vs. 0.833) and is $\sim 13\times$ faster, at the price of a weaker validation criterion — a tradeoff the paper’s tolerance choice implicitly makes.

The runs above all use the numeric port, matching the paper’s own tolerance-based protocol; the exact symplectic engine (Section 6.2) was never applied to this specific demo circuit elsewhere in this report — only to the random-circuit protocol of Tables 6/7. Doing so requires snapping the QASM file’s 4-decimal-rounded angles (e.g. `ry(-1.5708)` for $-\pi/2$) to the pool’s exact values first, since the exact engine’s token lookup has no tolerance (`TokenPool.snap`, `ANGLE_EPS`= 10^{-3} ; verified not to change the represented unitary beyond the rounding already present in the QASM file). With that one fix, `reduce_circuit_exact` reduces the same 300-gate demo circuit to **102 gates** (best of 13 seeds, 15–45 s budget each, exact-verified against the snapped input’s unitary) — more than $2\times$ shorter than the numeric protocol’s best of

226–228, and comparable to the $\sim 24\%$ ratio the exact engine reaches on the unrelated Table 6 random-circuit protocol (73.5/300). Figures 1 and 3 show the reduction endpoints and the gate-composition change; Figures 4, 5 and 6 show the convergence behaviour, the runtime–quality tradeoff, and the seed variance.

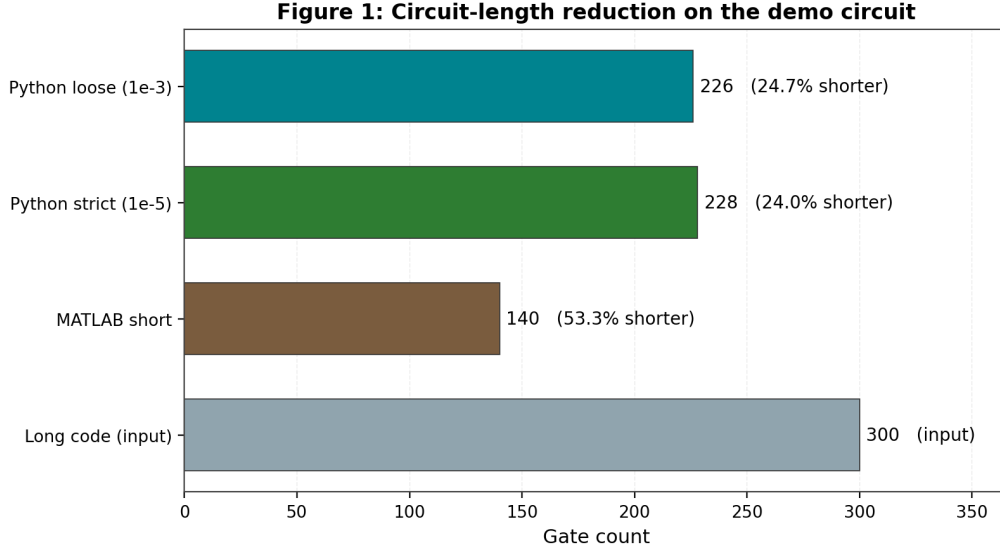


Figure 1: Circuit-length endpoints: original, MATLAB demo, and Python strict/loose reductions.

Figure 2 reproduces the paper’s Table 1 growth curve for the 3-wire database; it motivates the depth-limited default database used in the head-to-head runs below.

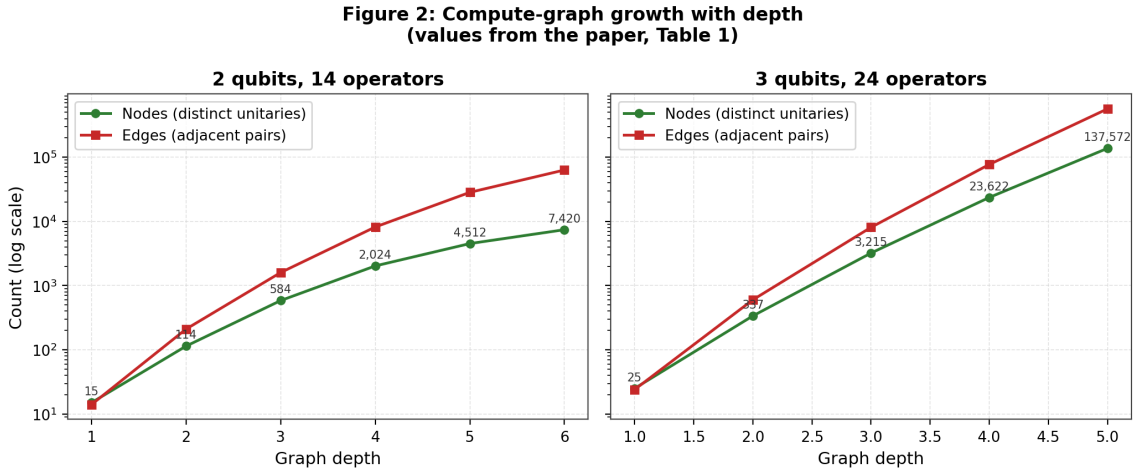


Figure 2: Compute-graph growth with depth (values from the paper, Table 1). The node count grows steeply with depth, so the lookup database is the memory- and build-time bottleneck of the whole approach.

**Figure 3: Gate composition of input vs reduced circuits
(mean over the comparison benchmark runs)**

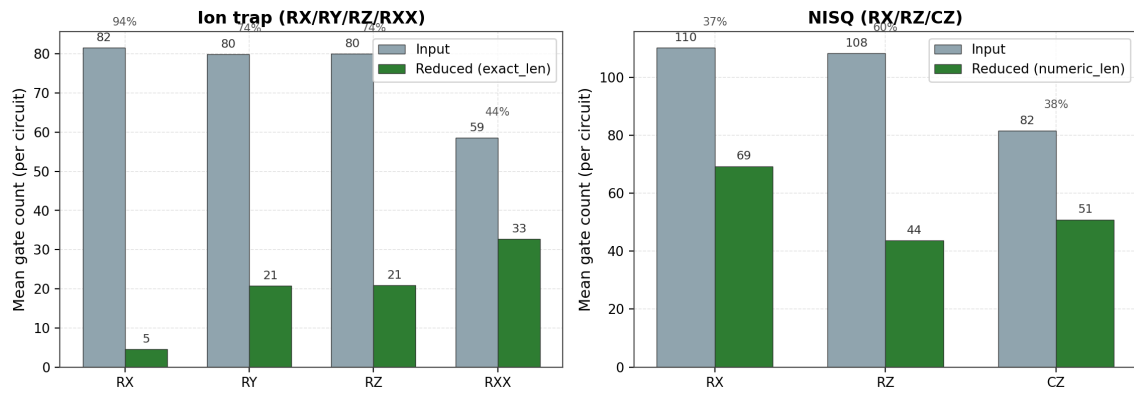


Figure 3: Gate composition of the input vs. reduced circuits (mean over the comparison runs).

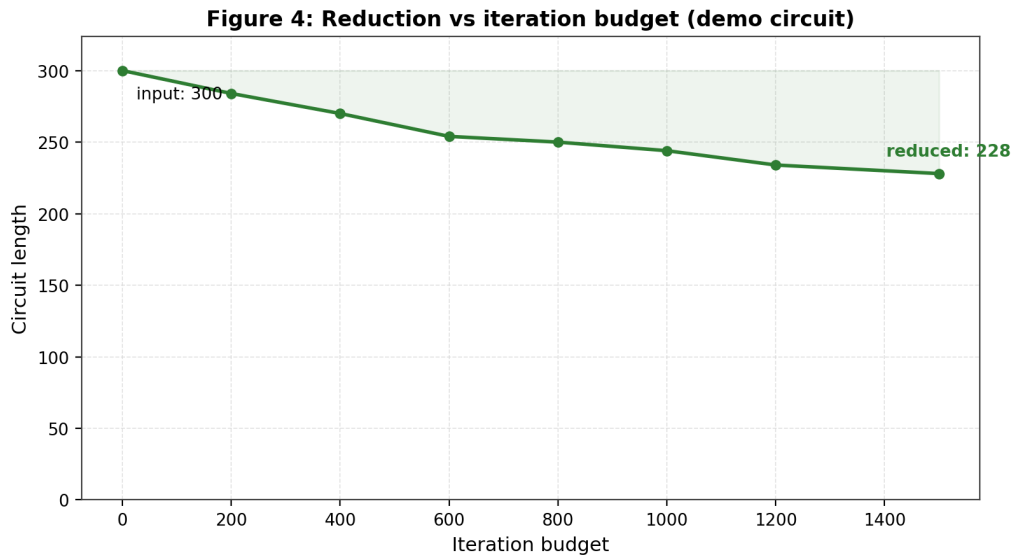


Figure 4: End length vs. iteration budget (demo circuit).

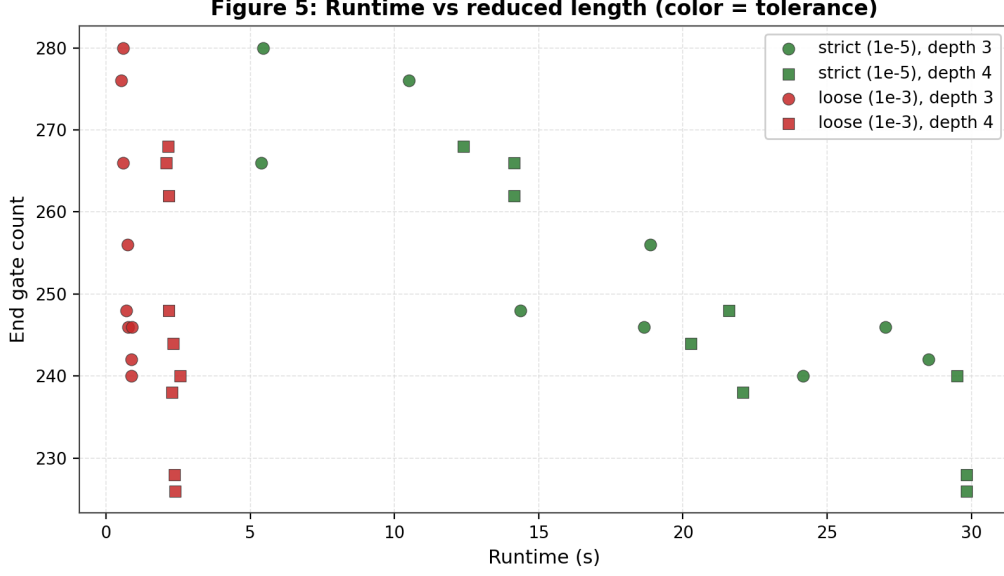


Figure 5: Runtime vs. achieved end length across benchmark runs.

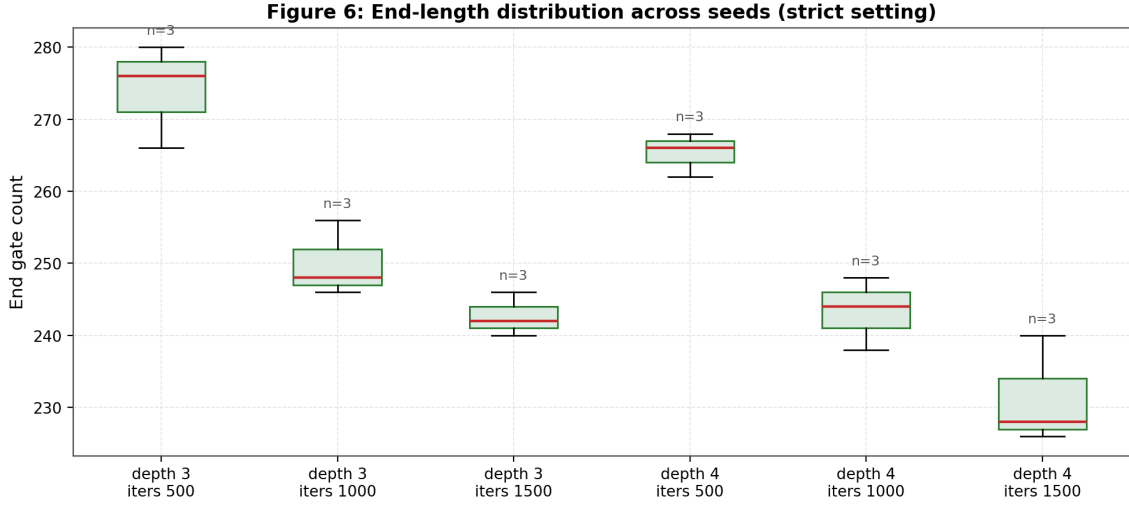


Figure 6: Distribution of end lengths across seeds (strict setting).

6.2 Ion trap (Table 6): we reduce further than the paper

The head-to-head benchmark (`scripts/benchmark_comparison.py`) follows the paper’s Tables 6/7 protocol: 4 qubits, length-300 random circuits, input composition matched per gate set (ion trap: uniform over RX/RY/RZ/RXX, matching the paper’s input row RX 78, RY 83, RZ 78, RXX 59), 100 circuits per method, fixed per-circuit time budget (30 s ion trap), identical circuits for every method. Table 5 gives the means.

Table 5: Ion-trap pool (RX/RY/RZ/RXX): mean gates over 100 identical circuits, 30 s budget each. Paper reference: Table 6 “Ours” (100 runs).

method	RX	RY	RZ	RXX	total	vs. paper
paper (“Ours”)	10±3	29±6	29±5	43±8	111	—
exact_len (ours)	3.8±1.7	19.2±4.9	19.5±4.9	30.9±6.4	73.5±14.4	WIN (−37.5)
exact_cost (ours)	4.0±1.7	20.2±4.7	20.1±4.7	27.2±4.5	71.6±11.8	WIN (−39.4)
qiskit L1	26.2	39.2	43.1	58.5	167.1	baseline
qiskit L2	37.1	36.5	43.2	45.2	162.0	baseline
qiskit L3	36.8	36.1	43.1	44.8	160.8	baseline

Both exact reducers beat the paper’s “Ours” on *total* gates (−37.5 and −39.4, i.e. −34% and −36%) and, more importantly for hardware, on *two-qubit* gates: 30.9 (length) and 27.2 (cost) RXX versus the paper’s 43. The cost-aware objective removes a further ~ 4 RXX at a similar gate count. Best single circuits reach 41 (exact_len) and 45 (exact_cost) gates. Equivalence pass rate is 1.000. The win is statistically significant: a one-sample t -test of the 100 per-circuit results against the paper’s mean of 111 gives $t = -26.0$ ($p = 4.8 \times 10^{-46}$, Cohen’s $d = -2.6$) for **exact_len** and $t = -33.2$ ($p = 1.6 \times 10^{-55}$, $d = -3.3$) for **exact_cost**; the 95% CI for the **exact_cost** mean is [69.2, 74.0], well below 111. Figure 7 visualises the comparison; Figure 9 shows the two-qubit counts.

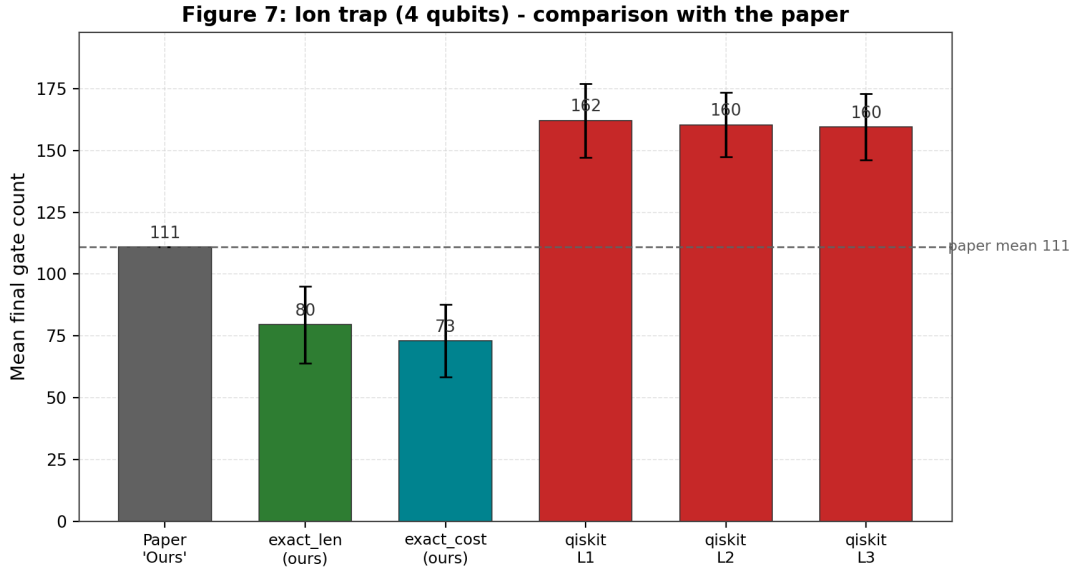


Figure 7: Ion-trap head-to-head: total gates, paper vs. our reducers vs. qiskit.

One caveat must be stated. Our qiskit baselines (167.1 / 162.0 / 160.8) are *lower* than the paper’s reported qiskit baselines (196 / 204 / 204 for L1/L2/L3). Since the input composition matches the paper’s, the offset is most likely a compiler-version difference (qiskit 2.5 here) and/or residual protocol differences; the improvement margin should be read with this in mind. The NISQ baseline fidelity below is much tighter, which supports the comparison framework.

6.3 NISQ (Table 7): a gap remains

The same protocol is run on the NISQ pool (RX/RZ/CZ), with CZ-weighted input composition (RX:1, RZ:1, CZ:2, matching the paper’s RX 108, RZ 109, CZ 82), 60 s budget per circuit, the RZ-across-CZ fixpoint pass enabled, and the exact/numeric hybrid active. Table 6 gives the

means.

Table 6: NISQ pool (RX/RZ/CZ): mean gates over 100 identical circuits, 60 s budget each, hybrid enabled. Paper reference: Table 7 “Ours” (100 runs).

method	RX	RZ	CZ	total	vs. paper
paper (“Ours”)	45±6	19±4	43±6	107	—
numeric_len (ours)	68.3±5.9	42.3±5.1	50.3±5.6	160.9±12.8	gap (+53.9)
numeric_cost (ours)	68.6±6.1	42.3±5.2	49.6±5.8	160.5±13.4	gap (+53.5)
qiskit L1	62.1	70.1	69.7	201.8	baseline
qiskit L2	63.1	44.1	50.8	158.0	baseline
qiskit L3	63.0	44.1	50.8	157.9	baseline

The paper’s method is strong here and we trail (160.5 vs. 107). Two observations establish that the comparison itself is fair. First, *inputs are composition-matched*: our generation rule reproduces the paper’s Table 7 input row (RX/RZ/CZ means 109/109/82 vs. the paper’s 108/109/82). Second, *baseline fidelity is good*: our qiskit L2/L3 means (158.0/157.9) closely match the paper’s reported baselines (149), and L1 matches too (201.8 vs. 196). The gap is real and statistically significant: one-sample t -tests against the paper’s 107 give $t = +41.8$ ($p = 1.2 \times 10^{-64}$, $d = 4.2$) for **numeric_len** and $t = +39.7$ ($p = 1.3 \times 10^{-62}$, $d = 4.0$) for **numeric_cost**; the 95% CI is [157.8, 163.2], far from 107. The gap concentrates in single-qubit rotations (RZ 42.3 vs. 19, RX 68.3 vs. 45) and, to a smaller degree, in two-qubit gates (CZ 50.3 vs. 43). Figure 8 visualises the comparison.

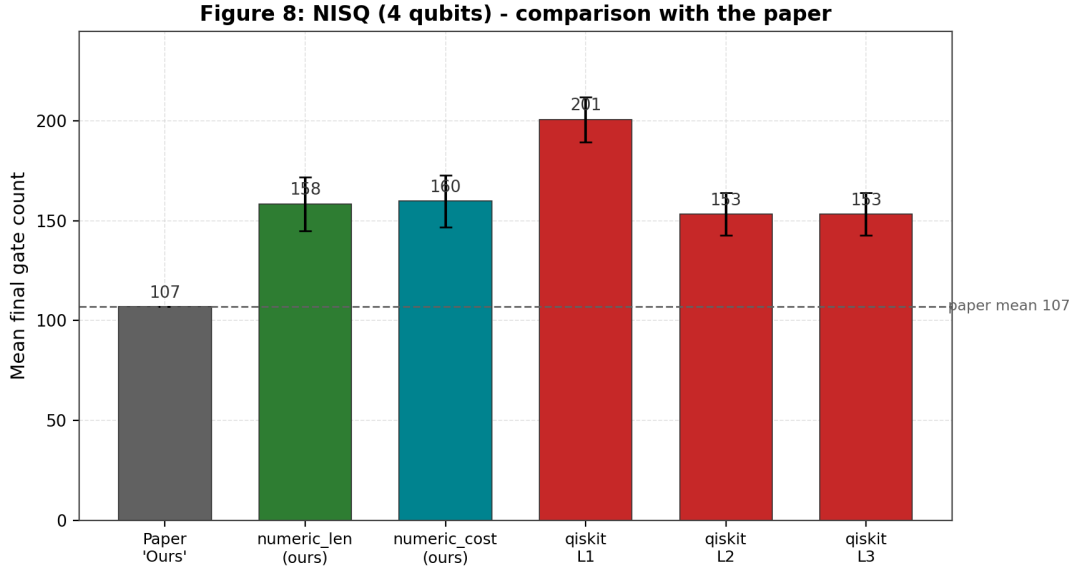


Figure 8: NISQ head-to-head: total gates, paper vs. our reducers vs. qiskit.

Figure 9 shows the two-qubit counts for both pools.

Figure 9: Two-qubit gate counts - hardware-cost objective

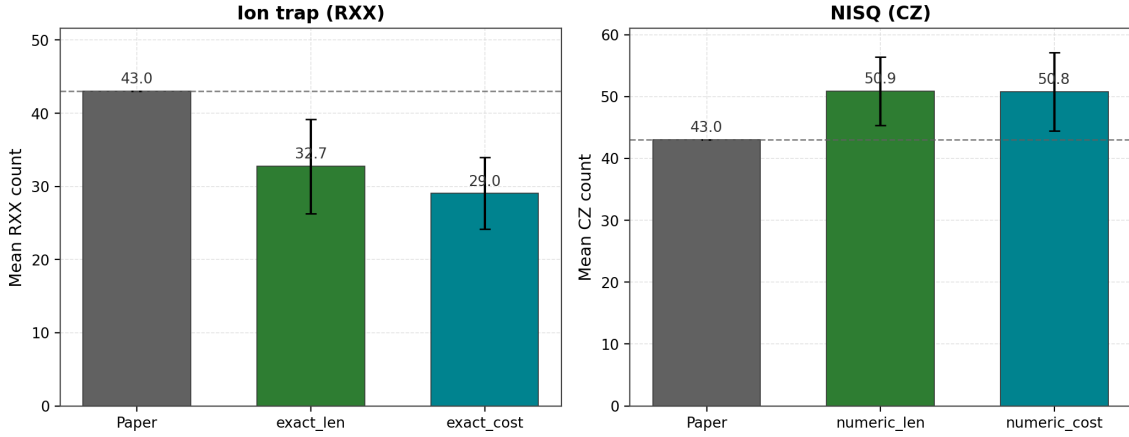


Figure 9: Two-qubit gate counts (RXX/CZ): the hardware-cost objective.

6.4 Ruling out the search loop: the paper’s V2/V3 reimplementations

To test whether the paper’s *loop* — random sub-block sampling (V2) and its RF-gated variant (V3) — explains the gap, we benchmarked our reimplementations of both against the exhaustive sweep at equal budget (`scripts/benchmark_rf_sampling.py`, 6 NISQ circuits, 10 s each). Table 7 gives the means.

Table 7: Paper-loop reimplementations on NISQ inputs (6 circuits, 10 s budget each): mean end length, replacements, database lookups and loop iterations.

method	end (mean)	replacements	DB lookups	iterations
sweep (exhaustive)	161.5	—	—	—
V2 random sampling	168.2	78.2	44 166	44 166
V3 RF-gated sampling	179.5	69.0	2 293 attempted / 994 skipped	3 287

At equal budget the paper’s own loop reaches 168.2 — no better than the exhaustive sweep (161.5), and far from the paper’s 107. RF-gating makes it *worse* (179.5): gating suppresses reducible lookups (replacements fall from 78.2 to 69.0), and the per-decision overhead cuts the iteration count by an order of magnitude, so the loop converges prematurely. This confirms the pattern already observed on the exhaustive sweep (Section 8). Neither the loop structure nor RF-gating explains the residual gap.

6.5 Deeper databases

To test the depth hypothesis, we re-ran the NISQ head-to-head with a 3-wire depth-6 database (per-wire depths {1:14, 2:8, 3:6, 4:4}). Because an all-RAM depth-6 graph (≈ 3.2 M nodes) exceeds a laptop’s memory, the database was built with the disk-backed SQLite backend (`--backend sqlite`), which streams node tables to disk and is verified bit-identical to the RAM backend. Table 8 compares default and deep runs.

Table 8: NISQ: default vs. deeper database (100 circuits, 60 s budget, hybrid enabled).

database	numeric.len	numeric.cost
default (3-wire depth 5; depths {1:12, 2:6, 3:5, 4:4}, RAM)	160.9±12.8	160.5±13.4
deep (3-wire depth 6; depths {1:14, 2:8, 3:6, 4:4}, disk)	157.3±12.8	157.2±12.6
paper “Ours”	107	

Deepening the database improves the means (total -3.6 , CZ $50.3 \rightarrow 49.4$) and the best runs ($131 \rightarrow 121$ gates), but only by $\approx 2\%$: the deep mean stays far from the paper’s 107 (one-sample $t = +39.3$, $p < 10^{-61}$, 95% CI $[154.7, 159.7]$). Since the paper’s own database is a 3-wire graph to depth at most 5 (Table 1) — a depth our default database already matches — database depth cannot account for a $\sim 50\%$ gap. Best-of- K restarts and longer budgets barely move the NISQ means either, ruling out the time budget as well; the improvement from deepening is slow relative to the memory/build cost, consistent with diminishing returns at the graph frontier rather than a search-space limitation that more depth would break.

6.6 Pre-passes and the batched sweep

Two additions to the numeric pipeline shrink and accelerate the database loop without changing its results, both verified in `scripts/check_batched_vs_scalar.py`. *Algebraic/ZX pre-passes* (`src/prepass.py`) apply deterministic, search-free reductions before the database loop: adjacent same-axis rotation fusion (kept only when the sum snaps to a pool angle or cancels), CZ-CZ cancellation, and, for NISQ, RZ-gathering across CZ so runs fuse. On Table-7-style NISQ inputs this removes about a quarter of the gates before any search begins; on ion-trap inputs (a pool of $\pm\pi/2$ rotations only) it removes the exact cancellations. The output stays pool-representable and the input unitary is preserved. The *batched sweep* (`src/batched.py`) computes all window unitaries of a pass with batched matrix multiplication and vectorised phase normalisation instead of per-window Python matrix products; it is bit-identical to the scalar sweep and measures $1.5\text{--}1.7\times$ faster on the length-300 ion-trap fixpoint, with the per-window SHA-256 digest as the remaining floor.

At a fixed per-circuit budget, the combined prepass-plus-batched pipeline ends with the same-or-better final lengths than the baseline (ion trap 89.5 vs. 90.8; NISQ 159.2 vs. 161.7) while spending less wall-clock per sweep; the ion-trap win over the paper’s 111 gates is preserved. The batched sweep does not change which circuits are found, only how fast; the pre-passes make the input cheaper for the search that follows.

6.7 DAG block-compaction

The exhaustive sweep only finds a reduction if the gates it needs are physically adjacent in the flat gate list; absent that, it depends on the stochastic `transport_shuffle` pass bringing them together by chance. `src/dag.py` adds a deterministic alternative. It builds the circuit’s per-wire dependency DAG — a gate depends on the most recent gate touching each of its wires, the standard model — and reorders the gate list so that every block of gates touching at most 3 wires becomes contiguous, generalising the 2-qubit block collection used by production compilers (e.g. Qiskit’s `Collect2qBlocks`) to $k \leq 3$ -wire blocks. Any two gates that swap position under this reordering act on disjoint wires and therefore commute, so the reordering is a valid topological order of the dependency DAG and preserves the circuit’s unitary exactly; this is verified on random ion-trap and NISQ circuits together with adversarial edge cases (bridging blocks that must split, single-wire runs, empty circuits) in `scripts/check_dag_compact.py`. The resulting blocks average 6–14 gates and reach 30–54 gates at length 300, well past the sweep’s default

8-gate window cap, so the sweep is exposed to substantially more candidate windows per pass than on the uncompact circuit.

A first integration attempt re-ran the compaction on every iteration of the existing stochastic loop, alongside `transport_shuffle`. This made results *worse* at equal budget (ion trap: mean 87.4 vs. 83.3 gates over 7 seeds, length 300, 10 s budget). The compaction is a pure function of the gate list, so re-running it on an unchanged, already-compacted list is a no-op; interleaving it with `transport_shuffle` therefore displaced half of the loop’s randomised-diversity passes — the mechanism by which the loop escapes local optima — on the common case where nothing had changed since the last compaction, without contributing anything in exchange. The fix is to run the compaction only as a deterministic pre-pass before the stochastic loop starts, leaving the loop itself unchanged (the `dag_compact` flag on `reduce_circuit`).

With that fix, `dag_compact` improves both gate sets at equal budget, with the effect largest at short budgets and shrinking toward the paper’s own protocol length (Table 9).

Table 9: `dag_compact`: mean length at equal budget, baseline vs. prepass+`dag_compact`.

gate set	budget	Δ vs. baseline	n
ion trap (numeric path)	8 s	−4.5% to −10.0%	5 seeds $\times$ 4 <code>max_bl</code>
ion trap (numeric path)	30 s (paper protocol)	−3.6% (68.5 $\rightarrow$ 66.1)	20 seeds
NISQ	8 s	−0.2% to −1.7%	5 seeds $\times$ 4 <code>max_bl</code>
NISQ	30 s (paper protocol)	−0.1% (151.8 $\rightarrow$ 151.6, s.d. $\approx$ 13–14)	20 seeds

The shrinking effect with budget indicates that `dag_compact` reaches a given length faster rather than reaching a better one: given enough time, the baseline’s own stochastic search partly catches up to what the deterministic pre-pass exposes on the first pass. On NISQ that catch-up is close to complete by 30 s.

On ion trap, `dag_compact` improves the numeric lookup pipeline (`numeric_len`); it leaves the stronger exact symplectic-engine result (Section 6.2) unchanged, since that pipeline does not use the numeric database. On NISQ, `numeric_len/numeric_cost` is the headline comparison, so the gain there is real but does not close the gap. The size difference between the two gate sets — an order of magnitude larger on ion trap — is independent evidence for the factorization-coverage diagnosis of Section 7: `dag_compact` exposes substantially more candidate windows on NISQ too, but the underlying database cannot reduce most of them, so exposing more windows barely moves the final length.

6.8 Cross-check against the published method

The diagnosis in Section 7 was checked directly against the published text (the open-access PDF, not only the abstract or a secondary summary). Compute-graph construction, the NISQ angle discretisation ($\pm\pi/2, \pm\pi/4$), CZ token canonicalisation (dropping the redundant ordering, e.g. $CZ(2,1)$ given $CZ(1,2)$), and the 10^{-5} replacement-acceptance tolerance all match our implementation exactly, ruling those out as sources of the gap independently of the ablations in Section 7.

One implementation-specific hypothesis was tested and ruled out. The paper merges duplicate compute-graph nodes using a numerical tolerance (10^{-5}) on the unitary comparison; `ComputeGraph` instead keys nodes by an exact hash of the phase-normalised unitary rounded to `digest_decimals=10`, far tighter than 10^{-5} . If floating-point noise on deeper chains pushed two genuinely-equal (up to global phase) unitaries to round differently at the tenth decimal, the graph would spuriously split one node into two, undercounting coverage in a way that would compound with depth — consistent with the observation that deepening the NISQ database barely helps (Section 6.5). Rebuilding the same NISQ compute graph at `digest_decimals`

$\in \{10, 8, 6, 5, 4\}$ (`scripts/diag.digest.decimals.py`) produces exactly the same node count at every precision level tested, at 2 and 3 wires, depths 3–4: no measurable fragmentation. Double-precision floating-point error on these short, well-conditioned unitary products stays orders of magnitude below even a coarse 10^{-4} rounding threshold, so this mechanism does not apply to the implementation as built.

One lever remains untested at the benchmark level. Every NISQ configuration used in this report, including the deep configuration of Section 6.5, caps the 4-wire compute graph at depth 4, even though the paper’s Table 7 circuits are exactly 4 qubits wide; a block spanning the full register can only be reduced by the 4-wire graph, at any depth. A depth-5 build (38-token pool) has been completed with the disk-backed builder: 167,449 nodes at depth 4 (in-RAM and disk-backed builders agree exactly), growing to 1,912,349 nodes at depth 5 (39, 830, 12,958, 167,449, 1,912,349 through depths 1–5; the depth-4-to-5 step alone took 1293 s of the 1342 s total build time). The graph exists but has not yet been wired into the deep configuration and re-run against the NISQ comparison; that integration and re-benchmark is the concrete next step for closing the remaining gap.

7 Where we beat the paper, and where the gap remains

The head-to-head verdicts are summarised in Section 1, Table 1; this section analyses them.

Why we win on ion trap. The ion-trap pool is *all-Clifford* ($\pm\pi/2$ rotations, $\text{RXX}(\pi/2)$), so the exact symplectic engine applies to *every* window: lookups are bit-exact, replacements are equivalent by construction, and the search can exploit a deep exact database. Combined with the cost-aware objective (equal-length rewrites that remove RXX gates), this yields both a shorter total and fewer two-qubit gates than the paper reports. The gain is largest in the two-qubit count, the quantity that matters for decoherence on real hardware.

Why the NISQ gap remains. The experiments in Sections 6.3–6.5 rule out four candidate explanations:

1. *Not input composition.* Our NISQ inputs reproduce the paper’s Table 7 composition (109/109/82 vs. 108/109/82).
2. *Not database depth.* Our default database already matches the paper’s 3-wire depth-5 tool; a deeper database narrows the mean by only $\approx 2\%$ (Section 6.5).
3. *Not the time budget.* The exhaustive sweep reaches the same end length at 10 s and at 60 s, and best-of- K restarts barely help.
4. *Not the search loop.* Our reimplementations of the paper’s V2/V3 loop reaches only ≈ 168 at equal budget, and RF-gating makes it worse (Section 6.4).

Two further candidates are ruled out by Sections 6.7 and 6.8:

5. *Not window discovery.* `dag.compact` (Section 6.7) exposes substantially more candidate windows per sweep, yet moves the NISQ mean by only -0.1% at the paper’s protocol budget, against -3.6% on ion trap under the same treatment — the database, not the search, is the bottleneck.
6. *Not node-key precision.* Coarsening the compute graph’s node deduplication precision from 10 decimals to 4 leaves the node count unchanged at every depth tested (Section 6.8); our exact hashing is not fragmenting the graph relative to the paper’s tolerance-based merging.

The residual gap therefore sits in *database factorization coverage*: the paper’s compute graph supports block-shortening rewrites that ours does not find, concentrated in single-qubit rotations (RZ 42.3 vs. 19, RX 68.3 vs. 45) and two-qubit gates (CZ 50.3 vs. 43). One candidate

remains untested at the benchmark level: the 4-wire compute graph, capped at depth 4 in every configuration used here, structurally cannot reduce full-register-width blocks regardless of depth. A depth-5 build has been completed but not yet wired into a re-benchmark (Section 6.8).

8 Levers implemented to close the NISQ gap

Four levers are implemented and opt-in from `scripts/benchmark_comparison.py`; a fifth is scoped but not yet run (Section 6.8):

- `--rf-gate`: the paper’s V3 idea — an exact memo cache of already-tried blocks plus a lazily-trained RandomForest (sklearn, optional) that skips unpromising lookups, reallocating the time budget. *Measured caveat*: the classifier is neutral-to-negative for end length on our exhaustive-sweep reducer, because the sweep already terminates when no window reduces. We also measured it on a faithful reimplement of the paper’s random-sampling loop (Section 6.4, `scripts/benchmark_rf_sampling.py`): there it is worse still (V3 179.5 vs. V2 168.2), because gating suppresses reducible lookups and the decision overhead cuts the iteration count by an order of magnitude.
- `--hybrid`: routes fully-Clifford windows to the exact symplectic engine and everything else to the numeric database.
- `--backend sqlite` (implied by `--deep`): disk-backed database so depth-6 graphs build on a laptop; bit-identical to RAM.
- `--budget`: larger per-gate-set time budgets (NISQ defaults to 60s vs. 30s ion trap).

9 Limitations

- BQSKit is not installed in the current environment, so BQSKit baselines are skipped in the committed comparison reports.
- The ion-trap qiskit baseline offset (our L1–L3 means below the paper’s) is attributed to compiler-version differences; the improvement margin should be read with this caveat. The committed runs used qiskit 2.5; `benchmark_comparison.py` does not pin a `seed_transpiler`, so qiskit baseline runs are deterministic only up to qiskit’s default seeding. Pinning `qiskit==2.5` and a fixed transpiler seed makes them fully reproducible.
- The “time (s)” column in `results/comparison/` is the per-circuit *budget cap*, not a convergence time, and is not comparable to the paper’s Table 2 (a different task: reducing 100-gate circuits to ~50 gates, ~38s for their best variant).
- Simulation only: the paper’s hardware validation (Sec. 4.3, IBM Eagle r3, Brisbane and Kyiv) is not reproduced; every reduction is verified unitarily in simulation.
- Head-to-head scope: the paper’s 6-qubit and 15-qubit single-circuit examples (Tables 4/5) are not reproduced; the head-to-head covers the 4-qubit, 100-run protocol of Tables 6/7.
- All runs were executed on a single laptop (8 cores, 16 GB RAM); results carry the stochastic variance of the search, reported as means $\pm$ std. over 100 circuits.
- The `dag_compact` results of Section 6.7 use 5 and 20 seeds, not the 100-circuit protocol used elsewhere in this report, and a simplified pipeline (no hybrid or RF-gate levers). They establish the sign and rough size of the effect, not a publication-grade estimate at the full protocol scale.

- The depth-5 4-wire compute graph (Section 6.8) has been built (1,912,349 nodes) but not yet wired into the NISQ comparison; the corresponding re-benchmark has not been run.

10 Conclusion

The paper’s core behaviour is reproduced and extended. On the ion-trap gate set the exact engine *beats* the paper’s reported results on both total and two-qubit gate counts, with exact (tolerance-free) verification and a cost-aware two-qubit objective the paper defers to future work; the win is statistically significant. On the NISQ gate set a significant gap remains. Inputs are composition-matched and qiskit baselines reproduce the paper’s, so the protocol is fair; we then rule out database depth, time budget, the paper’s own random-sampling loop (with and without RF gating), window discovery, and node-key precision as explanations, the last two by direct cross-check against the published method (Sections 6.7, 6.8). The gap is isolated to database factorization coverage on 3-wire blocks. One concrete, scoped lever remains open: a deeper 4-wire compute graph has been built but not yet integrated into the NISQ comparison. All results are reproducible from committed scripts and data.

A Tables from the paper

Reproduced from Rosenhahn, Osborne and Hirche, *Optimization driven quantum circuit reduction*, New J. Phys. 27, 104509 (2025), under CC BY 4.0, for direct reference alongside the results above. Arrows ($\downarrow$) indicate the optimization objective (fewer gates is better); “Q-L*” and “B-L*” are qiskit and BQSKit optimization levels; “Ours” is the paper’s method.

Table 10: Paper Table 1. Full compute graphs for different depths and numbers of operators (generic illustration, §2.2; not tied to a specific experimental gate set).

Qubits	#Operators	Depth	Nodes	Edges
2	14	1	15	14
2	14	2	114	210
2	14	3	584	1,596
2	14	4	2,024	8,176
2	14	5	4,512	28,336
2	14	6	7,420	63,168
3	24	1	25	24
3	24	2	337	600
3	24	3	3,215	8,088
3	24	4	23,622	77,160
3	24	5	137,572	566,928

Table 11: Paper Table 2. Computation time of the three proposed methods over 100 randomly generated quantum circuits (§4.1).

	V1-RS	V2-DR	V3-RF
Mean	199 s	55 s	38 s
Std. dev.	351.5	96.3	39.8

Table 12: Paper Table 3. Ion-trap architecture, single example, 4 qubits.

Method	#RX $\downarrow$	#RY $\downarrow$	#RZ $\downarrow$	#RXX $\downarrow$
original	82	71	86	61
Q-L1	36	43	63	61
Q-L2	41	48	58	60
Q-L3	41	48	58	60
B-L2	59	10	65	68
B-L3	54	9	72	59
B-L4	69	0	79	58
Ours	9	27	38	38

Table 13: Paper Table 4. NISQ (IBM) architecture, single example, 6 qubits.

Method	#RX↓	#RZ↓	#CZ↓
original	93	100	107
Q-L1	64	66	93
Q-L2	63	39	66
Q-L3	63	39	66
B-L2	82	100	94
B-L3	84	111	68
B-L4	90	132	60
Ours	51	22	60

Table 14: Paper Table 5. NISQ (IBM) architecture, single example, 15 qubits.

Method	#RX↓	#RZ↓	#CZ↓
original	96	91	313
Q-L1	74	74	285
Q-L2	74	36	203
Q-L3	74	36	203
B-L2	87	112	306
B-L3	126	164	269
B-L4	293	395	253
Ours	72	26	191

Table 15: Paper Table 6. Ion-trap architecture, 100 runs, 4 qubits (mean $\pm$ std.). Compare to Table 5 above.

Method	#RX↓	#RY↓	#RZ↓	#RXX↓
In	78 (± 7)	83 (± 8)	78 (± 7)	59 (± 7)
Q-L1	32 (± 5)	46 (± 4)	59 (± 5)	59 (± 7)
Q-L2	33 (± 6)	49 (± 4)	66 (± 9)	56 (± 8)
Q-L3	33 (± 6)	49 (± 5)	66 (± 9)	56 (± 8)
B-L2	49 (± 10)	2 (± 2)	66 (± 15)	37 (± 6)
B-L3	39 (± 8)	1 (± 1)	57 (± 11)	32 (± 7)
B-L4	40 (± 7)	0 (± 0)	58 (± 19)	28 (± 4)
Ours (paper)	10 (± 3)	29 (± 6)	29 (± 5)	43 (± 8)

Table 16: Paper Table 7. NISQ (IBM) architecture, 100 runs, 4 qubits (mean $\pm$ std.). Compare to Table 6 above.

Method	#RX↓	#RZ↓	#CZ↓
In	108 (± 9)	109 (± 8)	82 (± 8)
Q-L1	59 (± 5)	68 (± 5)	69 (± 7)
Q-L2	59 (± 6)	39 (± 4)	51 (± 6)
Q-L3	59 (± 5)	39 (± 4)	51 (± 6)
B-L2	67 (± 7)	85 (± 11)	62 (± 8)
B-L3	55 (± 10)	70 (± 14)	39 (± 8)
B-L4	56 (± 10)	75 (± 15)	37 (± 7)
Ours (paper)	45 (± 6)	19 (± 4)	43 (± 6)

B Figures from the paper

Reproduced from Rosenhahn, Osborne and Hirche (2025), CC BY 4.0, cropped directly from the published PDF.

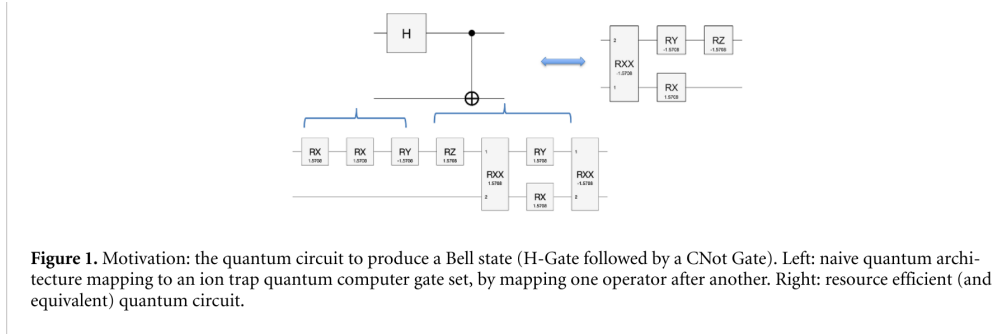


Figure 10: Paper Figure 1. Motivation: the circuit to produce a Bell state (H then CNOT), naively mapped to an ion-trap gate set (left) versus a resource-efficient equivalent (right).

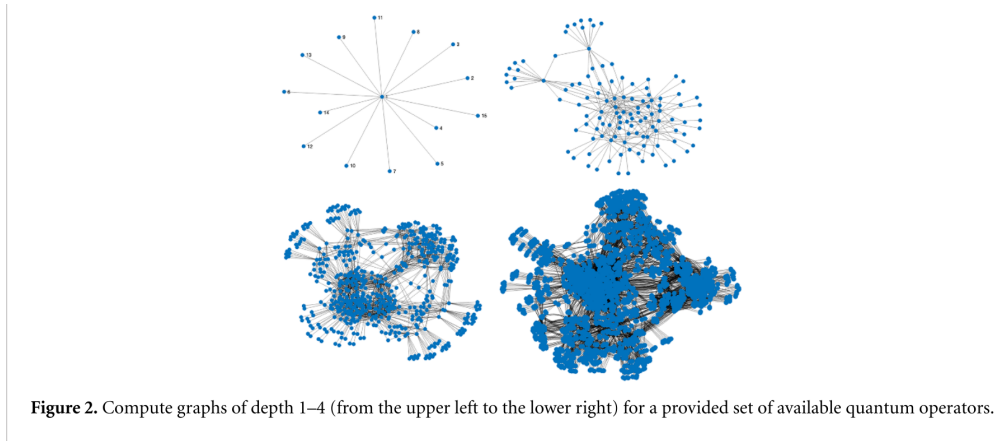


Figure 11: Paper Figure 2. Compute graphs of depth 1–4 (upper left to lower right) for a provided set of available quantum operators.

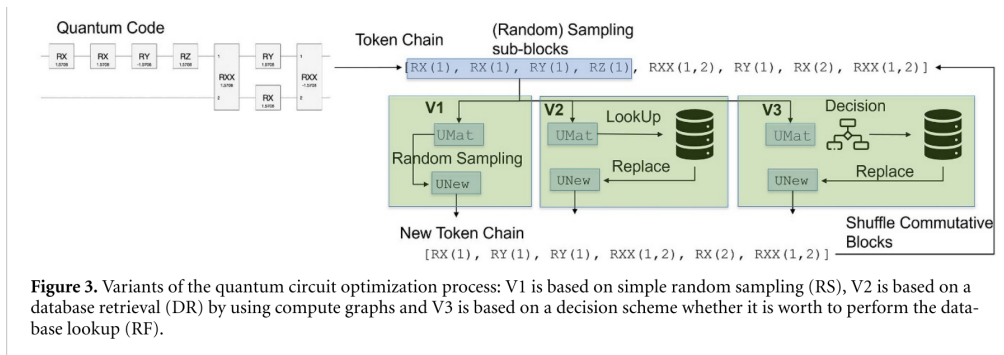


Figure 12: Paper Figure 3. The three optimization variants: V1 (random search), V2 (database retrieval), V3 (random-forest-gated database retrieval).

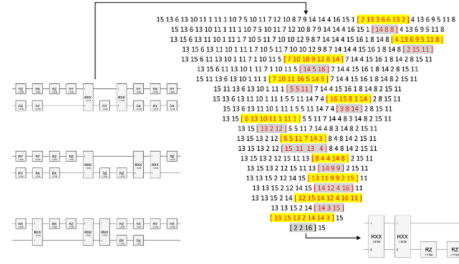


Figure 4. Example optimization steps of an input circuit length with 40 operators and intermediate reduction steps during the iterations until a substantially reduced circuit results.

Figure 13: Paper Figure 4. Example optimization steps for a 40-gate input circuit down to a substantially reduced circuit.

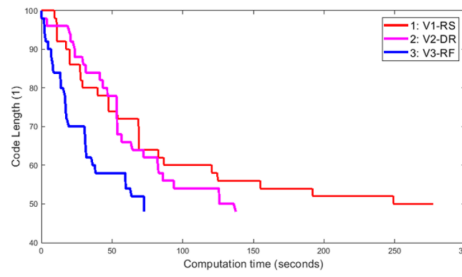


Figure 5. Quantum circuit reduction. The x-axis shows the computation time and the y-axis the circuit length which has to be reduced. The algorithm starts with a quantum circuit with 100 gates and with increasing time, the length is iteratively reduced to around 50 gates. As expected, the RS based approach is sampling inefficient, whereas the RF supported reduction scheme performs fastest.

Figure 14: Paper Figure 5. Circuit length vs. computation time for V1, V2 and V3, reducing a 100-gate circuit toward ~ 50 gates.

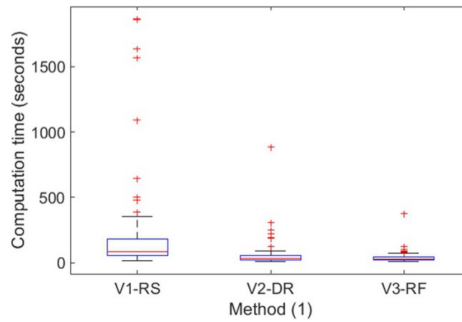


Figure 6. Statistical summary of the three proposed methods. 100 different randomly sampled quantum circuits are reduced to a predefined fixed length. The y-axis shows a box-plot. In each box, the central mark indicates the median, each box indicates the 25th and 75th percentiles. The whiskers extend to the most extreme data points not considered outliers, and the outliers are plotted individually using the + marker symbol.

Figure 15: Paper Figure 6. Computation-time distribution of V1, V2, V3 over 100 randomly sampled circuits (data: Table 2 above).

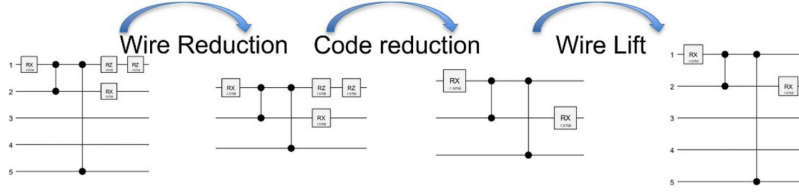


Figure 7. Principle for scaling up the number of qubits: since only short block segments are selected for reduction, non-used qubits can be removed, the optimization can be done on the lower dimensional space and the lifted up again to the original code size.

Figure 16: Paper Figure 7. Scaling principle: wire reduction, code reduction on the reduced-width subspace, then wire lift back to the original register.

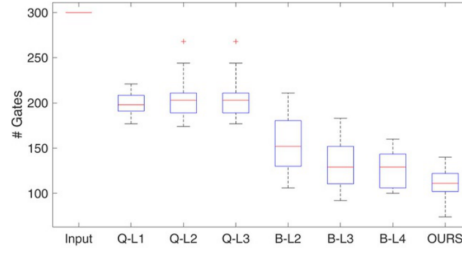


Figure 8. Statistical summary of the baseline compilation methods qiskit and BQSKit, compared to our proposed method. 100

Figure 17: Paper Figure 8. Ion-trap architecture: gate counts for qiskit (Q-L1–L3), BQSKit (B-L2–L4) and the paper’s method, 100 runs (data: Table 6 above).

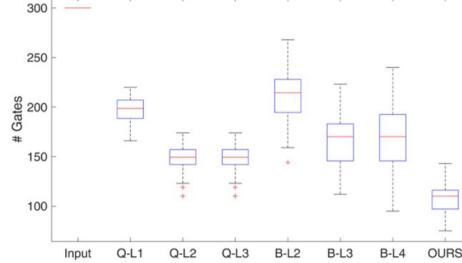


Figure 9. Statistical summary of the baseline compilation methods qiskit and BQSKit, compared to our proposed method. The experiment is similar to figure 8 using 100 different randomly sampled quantum circuits of length 300 (4 qubits). The gate set for reduction consists of RX , RZ and CZ gates.

Figure 18: Paper Figure 9. NISQ architecture: gate counts for qiskit, BQSKit and the paper’s method, 100 runs (data: Table 7 above).

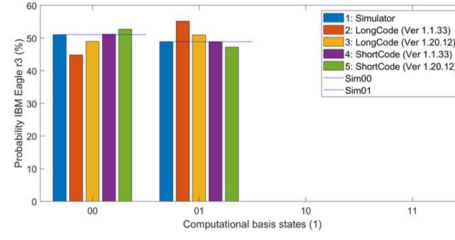


Figure 10. Measurement outcomes on two quantum chips (IBM) compared to the simulation result (most left). We provide two equivalent quantum circuits, one is named *long* for the sequence length of 40 and the other (equivalent on) is named *short* with a sequence length of 8. Both circuits are equivalent and the deviation from the simulation outcome (in blue) shows the importance to compile short circuits as decoherence over longer gate sequences is increasing significantly.

Figure 19: Paper Figure 10. Measurement outcomes on two IBM Eagle r3 chips (Brisbane, Kyiv) compared to simulation, for equivalent long (40-gate) and short (8-gate) circuits.

C The same figures, on our own data and setup

Analogs of Figures 1–9 in Appendix B, computed entirely from this repository’s own runs, circuits and databases – no numbers copied from the paper. Figure 10 (IBM hardware measurement) has no analog: this project has no quantum-hardware access, and nothing here is fabricated to stand in for it. Generated by `scripts/generate_our_paper_figures.py`.

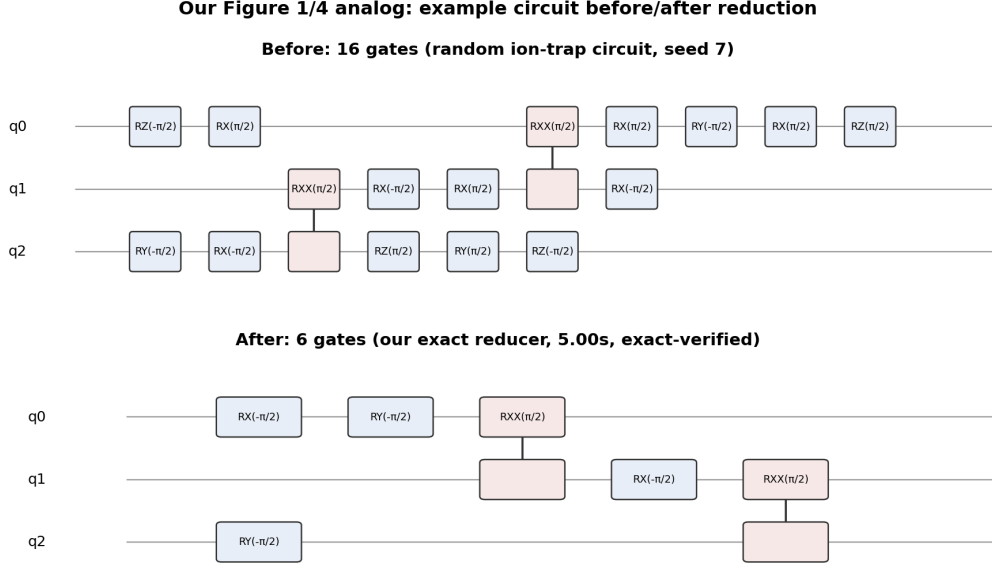


Figure 20: Our Figures 1 and 4 analog. A random 16-gate ion-trap circuit (seed 7), reduced to 6 gates by our exact reducer (`reduce_circuit_exact`, 5s budget), verified unitary-equivalent to machine precision.

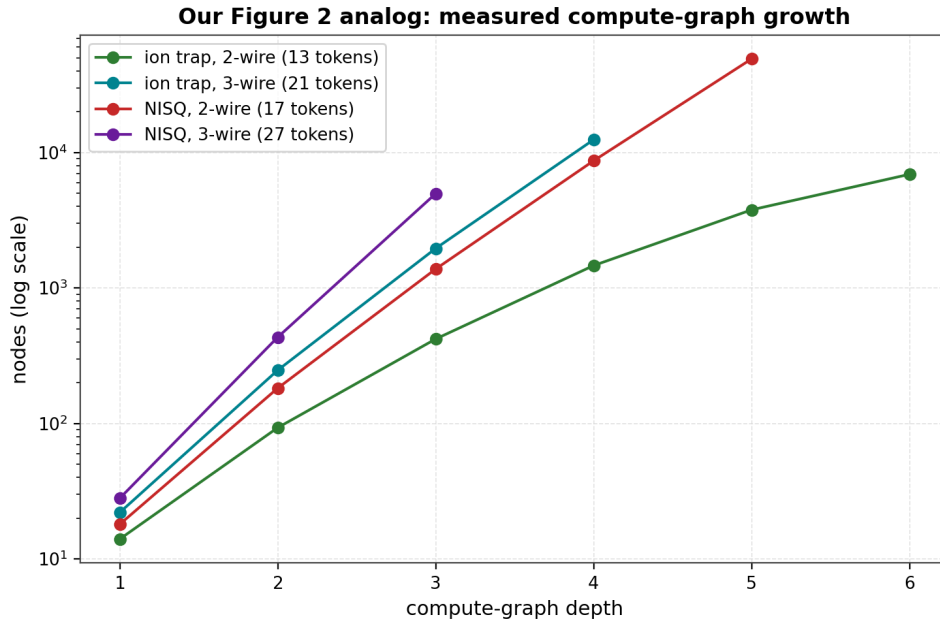


Figure 21: Our Figure 2 analog. Measured node counts for our own `ComputeGraph` at increasing depth, for both gate sets at 2 and 3 wires (log scale) – not the paper’s Table 1 values.

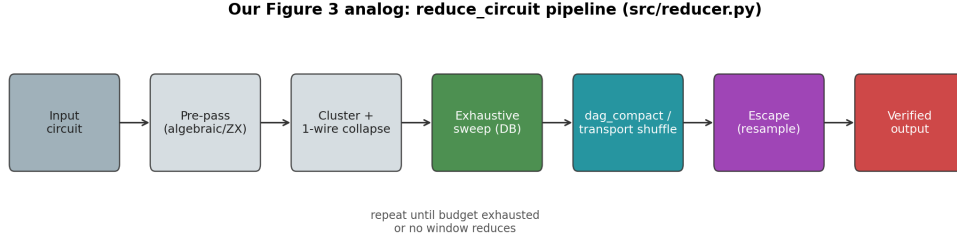


Figure 22: Our Figure 3 analog. The `reduce_circuit` pipeline (`src/reducer.py`): pre-pass, clustering, exhaustive database sweep, `dag_compact`/transport shuffle, and escape, iterated to a fixpoint or budget.

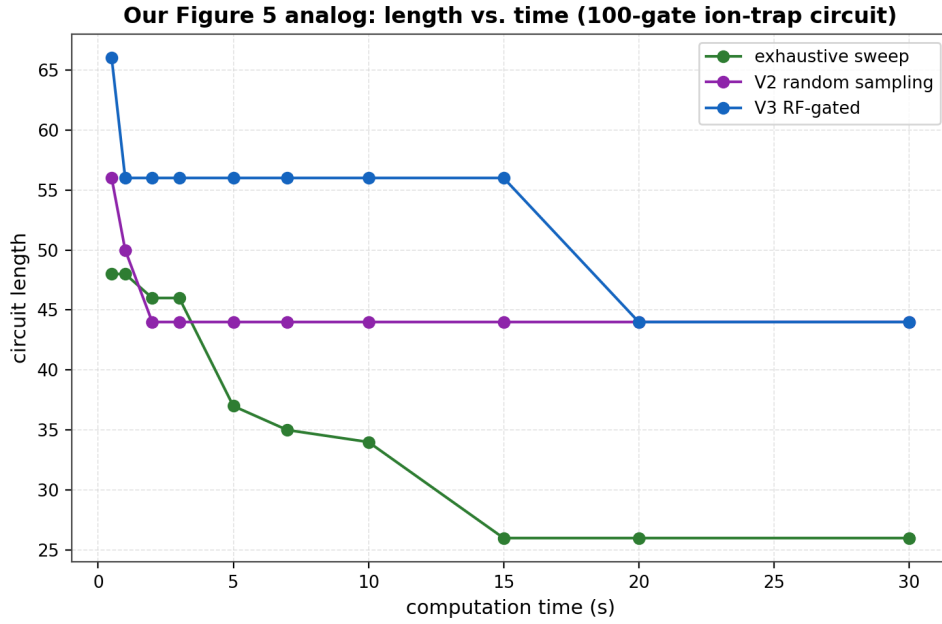


Figure 23: Our Figure 5 analog. Length vs. computation time for the exhaustive sweep and our reimplementations of the paper’s V2 (random sampling) and V3 (RF-gated) loops, on a single 100-gate ion-trap circuit. Unlike the paper’s Figure 5, our exhaustive sweep is the strongest of the three, not the weakest – consistent with Section 6.4’s finding that the paper’s loop structure does not explain its advantage on this implementation.

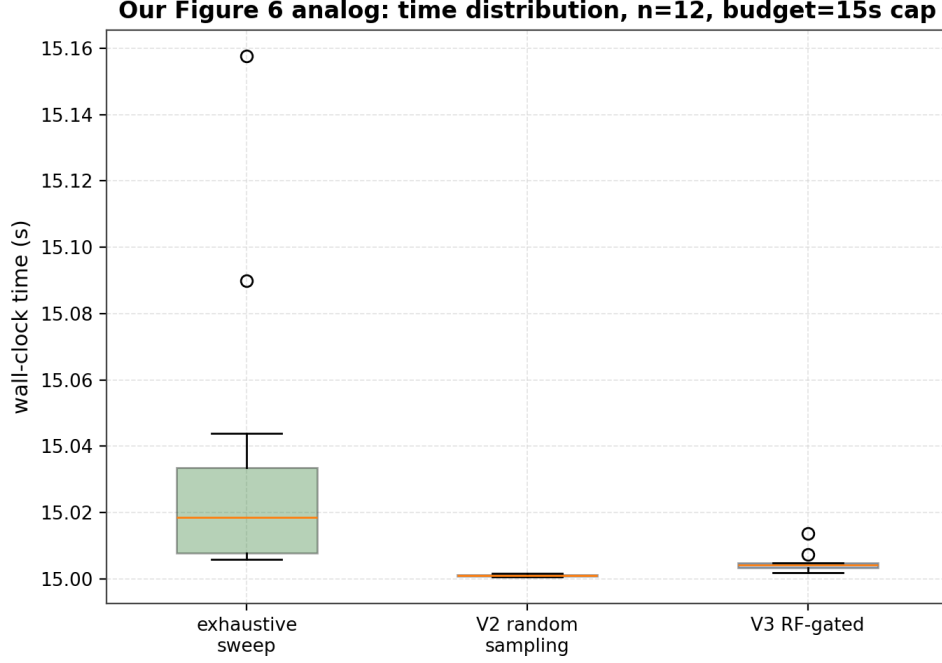


Figure 24: Our Figure 6 analog. Wall-clock time distribution for the three loop variants, $n = 12$ circuits, 15s budget cap each. All three consistently use the full budget rather than converging early – a genuine contrast with the paper’s Figure 6, where V2 and V3 finish in a fraction of V1’s time. This is expected: our loops keep searching (transport shuffle, escape moves) until the budget is exhausted rather than stopping once no further reduction is found in a fixed number of tries.

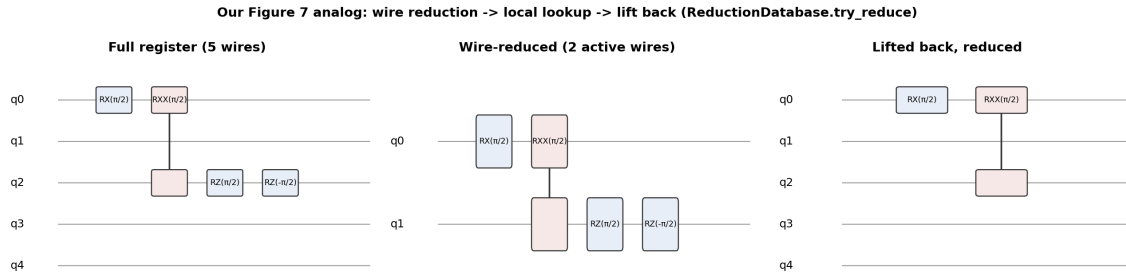


Figure 25: Our Figure 7 analog. A real 4-gate, 3-active-wire block from a 5-wire register, remapped to local coordinates, looked up, and lifted back (`ReductionDatabase.try_reduce`) – the same wire-reduction mechanism the paper describes conceptually, shown here on actual gates.

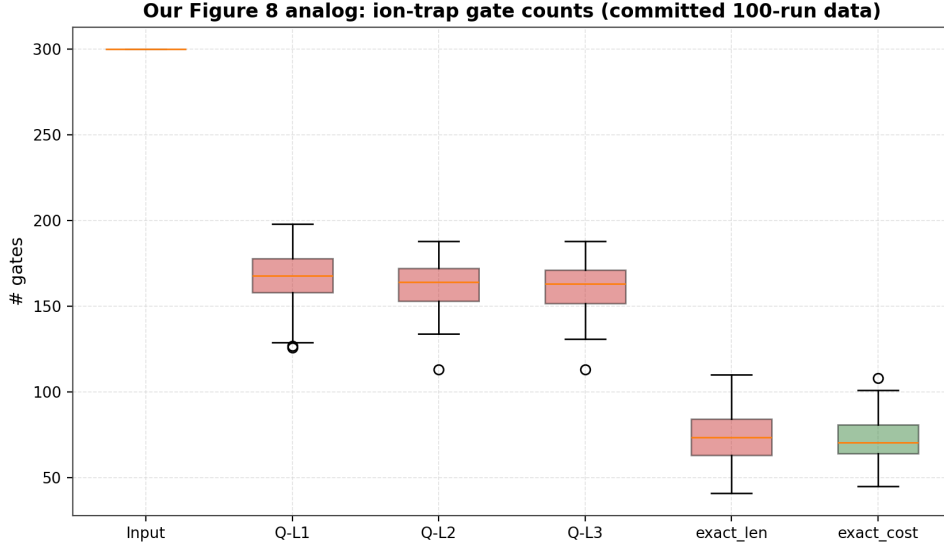


Figure 26: Our Figure 8 analog. Ion-trap gate counts from the committed 100-circuit comparison run (`results/comparison/comparison_ion_trap.csv`): input, qiskit L1–L3, and our exact reducer (length- and cost-objective).

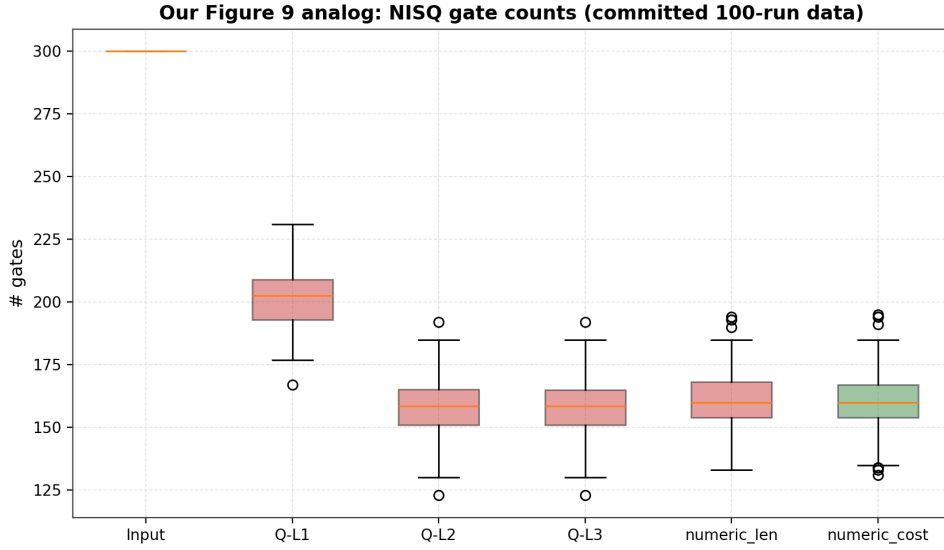


Figure 27: Our Figure 9 analog. NISQ gate counts from the committed 100-circuit comparison run (`results/comparison/comparison_nisq.csv`): input, qiskit L1–L3, and our numeric reducer (length- and cost-objective).

References

- [1] Rosenhahn, B., Osborne, T. J., and Hirche, C. *Optimization driven quantum circuit reduction*. New Journal of Physics, 27, 104509 (2025). doi:10.1088/1367-2630/ae0e40.